MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring

Phase Review 3

Michał Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
office@pads.rwth-aachen.de

Abstract. This document reviews Sprint 3 of the project, covering its objectives, technical implementation, and retrospective. Building on the time-series feature foundation established previously, this sprint focused on enriching and optimising the time-series feature set, adding dimensionality reduction and feature-pruning support, and implementing the evaluation, selection, and explainability machinery needed to compare baseline and time-series-enhanced models. We expanded the active-case-count feature into rolling-window statistics and vectorised their computation, wired configurable PCA and feature pruning into the pipeline, and added plateau-based prefix selection, model comparison, a feature-impact comparison tool, and permutation-based feature importance. The notebook suite was reworked to expose these capabilities through the same public API as the command-line pipeline. The empirical results and the baseline-versus-enhanced model comparison produced by this machinery are reserved for a dedicated evaluation document in the next phase.

Keywords: Predictive Process Monitoring · Process Mining · Time-Series Features · Feature Selection · Model Evaluation · Explainability

1 Introduction & Objectives

1.1 Sprint Goals

The goal of Sprint 3 was to make the pipeline ready to answer the project's central question — whether time-series information improves predictive process monitoring — by completing the feature and evaluation infrastructure required for a rigorous comparison. Concretely, the sprint aimed to (i) broaden the set of time-series features beyond a single active-case count and improve their computational efficiency, (ii) add mechanisms to manage feature dimensionality and prune uninformative features, (iii) implement the selection logic that identifies the most suitable prefix length and model for each task, and (iv) provide the explainability and comparison tooling needed to contrast log-only baselines with time-series-enhanced models. In line with the project's incremental approach,

this sprint delivers the *capability* to run these analyses; the resulting metrics and the model comparison itself are deferred to a separate evaluation document in the next phase.

1.2 Team Organization

Work continued to be tracked through GitHub Issues, with each track captured as a parent issue and individual tasks as linked sub-issues and pull requests, keeping the `dev` branch in a working state throughout the sprint. Michal coordinated the sprint and led the feature, model, and prefix selection tracks together with reporting and visualisation. Konstantinos implemented the time-series vectorisation, feature pruning, model-comparison, and notebook-revamp pull requests. Roman implemented PCA-based dimensionality reduction and added further log based features.

1.3 Role Allocation

Initial role allocation was done according to table 1, but with this sprint being the last one most roles responsibilities were already fulfilled and issues were assigned to people dynamically and without constraints.

Table 1. Role assignments for Sprints 1–3.

Role	**Sprint 1**	**Sprint 2**	**Sprint 3**
Project Coordinator / Scrum Master	Michal	Konstantinos	Roman
Technical Lead / Architecture	Roman	Michal	Konstantinos
Data & Preprocessing Lead	Roman, Konstantinos	Konstantinos	Roman
Modelling & Experimentation Lead	Michal	Roman	Michal
Evaluation & Visualization Lead	Konstantinos	Michal	Konstantinos
Quality & Tooling (Tests, CI)	Konstantinos	Roman	Roman
Documentation & Reporting	Michał, Roman, Konstantinos (shared)		

2 Technical Implementation

Sprint 3 extended the modular pipeline established in earlier sprints without restructuring it: every addition slots into the existing strategy-based stages defined in `data/schemas.py` and `data/types.py`. The following subsections describe the new time-series features and their vectorisation, the dimensionality-reduction and feature-pruning support, the model and prefix selection logic, the comparison and explainability tooling, and the supporting configuration and notebook changes. Table 2 maps each work item to its issues and pull requests and the modules it introduced or extended.

2.1 Time-Series Feature Expansion and Vectorisation

The time-series feature track was extended from a single active-case indicator to a richer set of derived signals. The `ActiveCaseCountFeature` computes the number of cases active at the timestamp of a prefix's last event, while `preprocess_time_series` builds a continuous hourly grid spanning the full log and `extract_time_series_features` computes rolling-window statistics over the active-case series. These features are aligned to case prefixes by timestamp through `align_features_to_prefixes`, preserving the strict temporal correspondence required to avoid leakage. The window and statistic computations, which were originally implemented with per-window Python loops, were rewritten using vectorised array operations, removing a significant bottleneck in feature extraction.

2.2 Dimensionality Reduction and Feature Pruning

Two complementary mechanisms for controlling the feature space were added and exposed through configuration. Principal Component Analysis is now an optional, configurable pipeline stage governed by a `pca_config` block (`active` and `keep_variability`), allowing a target fraction of retained variance to be specified per run. Independently, the feature-engineering configuration gained a `drop_features` list that removes named columns from the training and test matrices before model fitting; this provides a direct, reproducible way to prune features that prior importance analysis flags as uninformative. Together these support the "best feature set" selection workflow for both log-based and time-series features at the configuration level, without code changes.

2.3 Prefix and Model Selection Logic

Hyperparameter tuning is performed per model via `RandomizedSearchCV` in `search_hyperparams`, driven by each model's `param_grid` and the shared `search` configuration (iterations, cross-validation folds, optional search sample size). For prefix selection, `_detect_plateau_prefix` walks prefix lengths in ascending order and identifies the point at which the relative improvement of the tracked metric falls below a configurable threshold (default 5%), marking the prefix length beyond which additional events add little predictive value. `compare_models` aggregates the per-prefix metrics into a structured comparison that ranks models and records each model's plateau prefix, providing the basis for the "best model" and "best prefix" selection for each task.

2.4 Comparison and Explainability Tooling

To contrast log-only baselines against time-series-enhanced models, a dedicated command-line tool (`evaluation/evaluate_feature_impact.py`) loads the evaluation artifacts of two runs, combines them into a long-format table tagged by source (baseline versus advanced), and produces faceted, per-model plots of each

metric across prefix lengths. Explainability is provided by `evaluation/feature importance.py`, which computes permutation importance both globally and per prefix length and renders importance bar charts, per-prefix heatmaps, and importance trajectories. These tools generate the inputs for the forthcoming comparison; the analysis of their output is intentionally left to the next phase.

2.5 Configuration and Notebook Support

Supporting changes improved usability and reproducibility. The command-line pipeline now derives a default output directory from the configuration file path, so runs are written to predictable, config-specific locations. The notebook suite was reworked into a coherent set (`00_data_loading`, `01_exploratory data analysis`, `02_feature_analysis`, `03_model_evaluation`) that drives the pipeline through the same public API as the command-line entry point, demonstrating feature extraction, dimensionality analysis, and the loading of evaluation artifacts without bespoke wrappers.

Table 2. Sprint 3 implementation steps.

Step	Description	Issues / PRs
1	Expand and vectorise time-series features	#79, #87, #88
2	PCA-based dimensionality reduction	#76, #84
3	Feature pruning via configuration	#92
4	Plateau-based prefix and model selection	#80, #81, #90
5	Baseline-vs-enhanced comparison tool	#82, #93
6	Permutation feature importance	#73
7	Metric reporting and visualisation	#78, #91
8	Config-derived output paths	#95
9	Notebook revamp	#98 (#97)

3 Review & Retrospective

3.1 Successes

The sprint completed its planned scope. The pipeline now supports an enriched, efficiently computed time-series feature set, configurable dimensionality reduction and feature pruning, and a full selection-and-explainability layer, all integrated without disturbing the existing modular architecture. Table 3 lists the issues and pull requests closed during the sprint.

Table 3. Sprint 3 issues and pull requests.

Issue / PR	Description
#73	Implement feature importance
#74	Choose best time-series features
#75	Choose best log-based features
#76	Implement dimensionality reduction with PCA
#78	Visualize reports with metrics
#79	Add more time-series features
#80	Choose best prefix length
#81	Choose best model for both tasks
#82	Compare model performance with and without time-series features
#87	Vectorize time series and windows
#88	Vectorize time series and windows (PR)
#90	Choose best prefix and model (PR)
#91	Visualize reports with metrics (PR)
#92	Feature pruning (PR)
#93	Model comparison (PR)
#95	Choose default output path based on config path
#98	Revamped notebooks (PR, #97)

3.2 Difficulties

A recurring tension during the sprint was the separation between delivering selection and comparison *machinery* and producing the actual results: several "choose best" issues required the supporting evaluation code to be in place before any selection could be made, so the concrete choices of feature set, prefix length, and model are deferred to the evaluation phase. The exploratory data analysis notebook originally scoped as issue #89 was closed as *not planned* after its content was absorbed into the reworked notebook suite.

4 Moving Forward

4.1 Future Planning

With the feature, selection, and explainability infrastructure complete, the next phase shifts from implementation to analysis. The planned work is:

- **Evaluation and model comparison.** Run the baseline and time-series-enhanced configurations end-to-end and use the comparison and feature-importance tooling to assess whether time-series features improve predictive performance, analysing results across prefix lengths for both the outcome and remaining-time tasks.
- **Results document.** Produce the dedicated evaluation report presenting these findings, including the comparative analysis and feature-importance discussion.

– **Finalisation.** Consolidate documentation and the final report, and close all remaining open items.

References

1. van Dongen, B.F.: BPI Challenge 2017. Eindhoven University of Technology (2017). https://research.tue.nl/en/datasets/bpi-challenge-2017